

Video Game Programming: Game AI in Motion

L12

Prof. Fabio Buttussi, PhD
Human-Computer Interaction Laboratory
Department of Mathematics, Computer Science, and Physics
University of Udine - Italy

Artificial intelligence eras

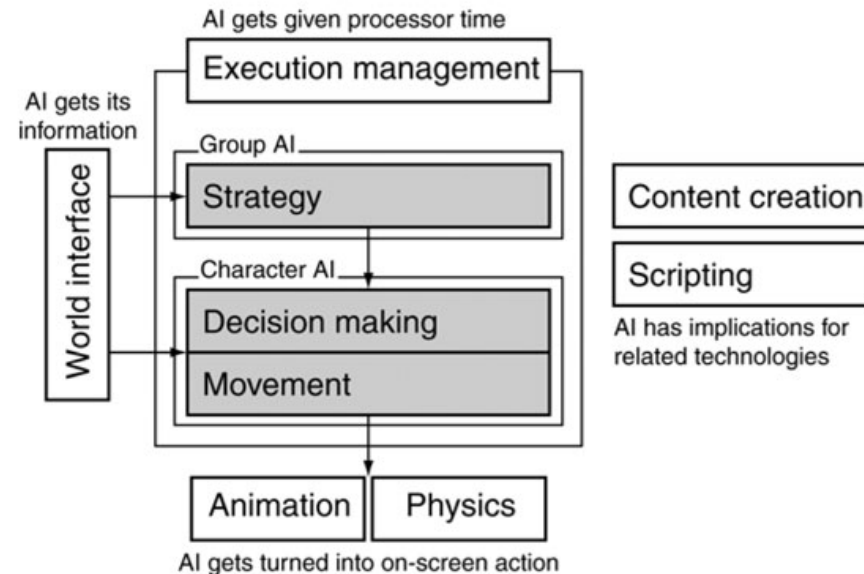
- Artificial intelligence (AI) tries to make computers able to perform human thinking tasks
- After early days during war, first AI era ('50-'80) concerned **symbolic systems** which consist of a set of knowledge and a reasoning algorithm manipulating symbols to create new knowledge (e.g., state machines, rule-based expert systems)
- After a decline, there was a move toward **natural computing / statistical techniques** (e.g., neural networks, genetic algorithms)
- Current era of **deep learning** and **generative AI** uses neural networks and new techniques on more powerful hardware

AI for games

- Some (synthetized) quotes from Millington, AI for games, 2019:
 - It is a common mistake to think that the more complex the AI, the better the characters will look. Creating good AI is all about matching game requirements to the right behaviors and algorithms.
 - Knowing when to be complex and when to stay simple is the most difficult element... The best AI programmers are those who can use a very simple technique to give the illusion of complexity.
 - Search and knowledge are intrinsically linked. The more knowledge you have, the less searching for an answer you need; the more search you can do, the less knowledge you need.
 - For AI designed to run on the device during the game, low computation / high knowledge approaches are often the clear winners. And these are very often symbolic.

Game AI model

- AI in most games addresses three basic abilities:
 - to move characters
 - to make decisions
 - to think tactically or strategically



AI model in Millington, AI for games, 2019

Abilities and infrastructure

- **Movement** refers to algorithms that turn decisions into some kind of motion
- **Decision making** involves a character working out what to do next
- **Strategy** refers to an overall approach used by a group of characters
- An additional **infrastructure** is needed to sense the world and implement decisions into actions
- **Agent-based AI** is about producing autonomous characters that take in information from the game data, determine what actions to take based on the information, and carry out those actions

Game AI in practice

- In practice, AI for a game consists in:
 - hackings
 - heuristics
 - algorithms
- **Hackings** are ad hoc solutions for the specific game
- **Heuristics** are rules of thumb that work in most, but not all, cases
- **Algorithms** are the generalizable part of the AI, which can be applied to different games

Common heuristics

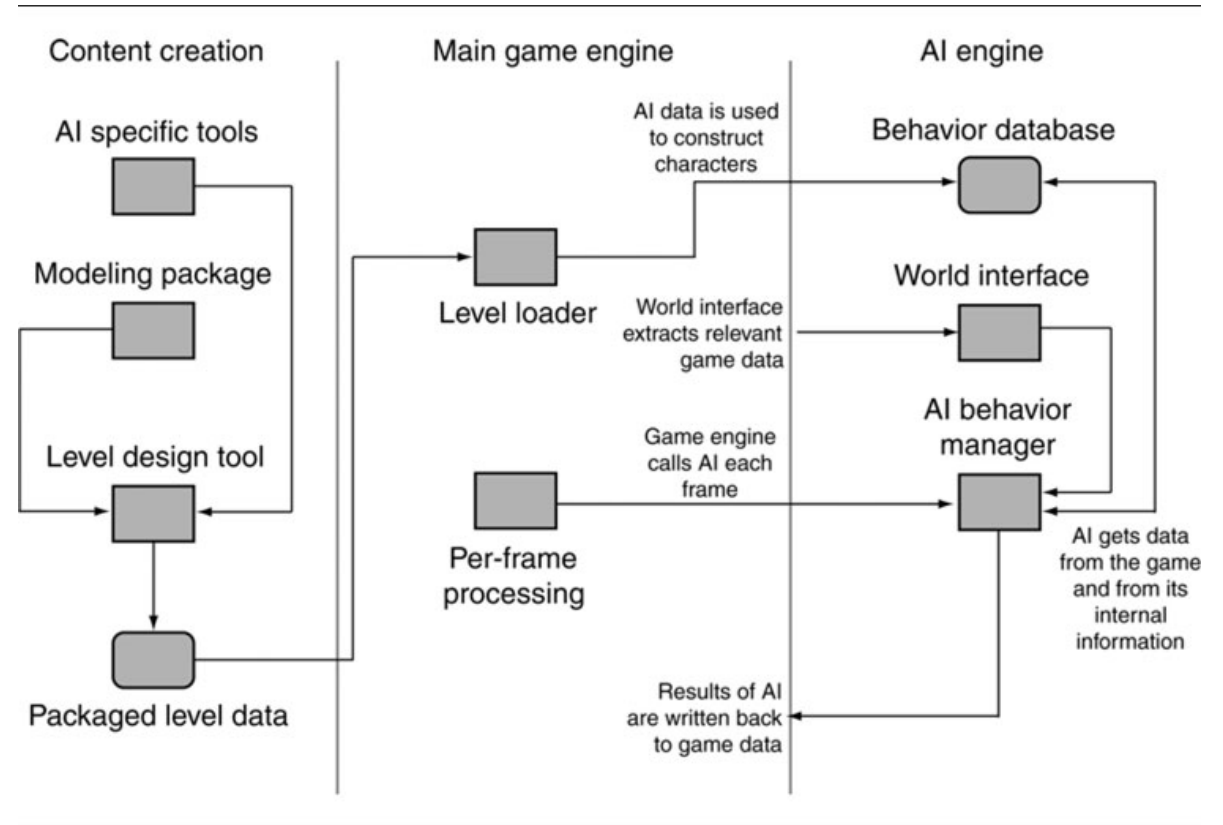
- *Most Constrained*: given the current state of the world, one item in a set needs to be chosen; the item chosen should be the one that would be an option for the fewest number of states
- *Do the Most Difficult Thing First*: the hardest thing to do often has implications for lots of other actions; it is better to do this first, rather than find that easy stuff goes well but is ultimately wasted
- *Try the Most Promising Thing First*: if there are several options open to the AI, it is often possible to give each one a score; even if it is inaccurate, trying the options in decreasing score order will provide better performance than trying things purely at random

AI constraints

- There are usually more challenging constraints in AI algorithms for games than for other AI applications:
 - CPU and GPU that are used also for graphics, physics,...
 - Memory that is also used for textures,...
 - Differences between platforms and within a platform
- A main difference with graphics is that **AI does not scale**
- A widely used solution is to target AI at the minimum specification machine listed in the technical design document

AI tools

- AI is algorithms and knowledge
- After implementing algorithms knowledge data must be modeled
- Knowledge data are usually created with tools
- At runtime, the AI behaviors are created applying algorithms to knowledge data



Movement

- Movement is fundamental in most games
- In the AI model, movement is the lowest level, the closest to actual actions
- Movement algorithms include chasing enemies, following paths, and avoiding collisions
- **Kinematic** algorithms output direction (and velocity), while **dynamic** algorithms output acceleration
- Non-kinematic, dynamic movement was called **steering behavior** in Reynolds, Steering behaviors for autonomous characters, Procs. Game Developers Conf., 1999, pp. 763-782

2½D movement

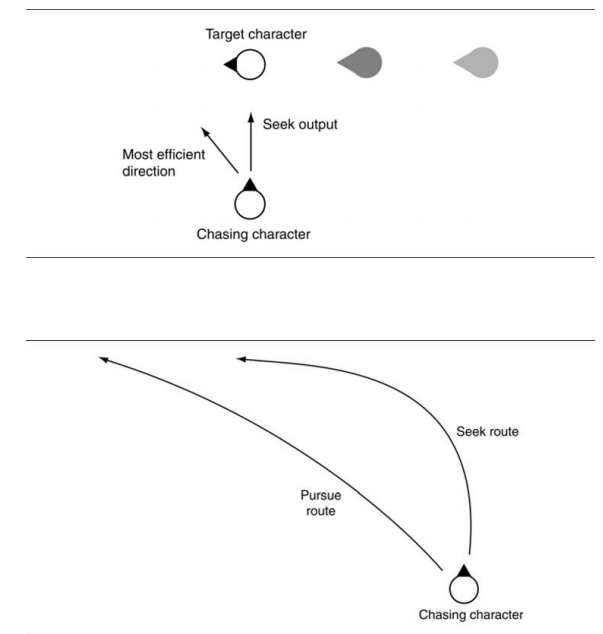
- Most 3D games have characters under the influence of gravity, constraining their movement to two dimensions
- Many movement algorithms treat a character as a single point
- Size is used for collision avoidance
- A character has thus 2 coordinates (x and z) and an orientation (y)
- 2½D is about 2D movement plus jump or other kinds of movement on y axis (without orientation change on x and z)
- 3D movement with 6DOF adds complexity and is used in a small subset of games (e.g., sci-fi games with spaceships)

Variable matching steering behaviors

- **Variable matching**, i.e., trying to match one or more elements of a kinematic target (e.g., same position, orientation, velocity)
- Several variable matching behaviors have a natural opposite
- **Seek / flee** tries to match / get far from target position by finding direction and heading at maximum speed (gradually increasing it)
- **Arrive** is like seek, but slowing down to exactly match the target using two radius (to decide when it is arrived and to slow down)
- **Align** tries to exactly match target orientation
- **Velocity matching** tries to match target velocity

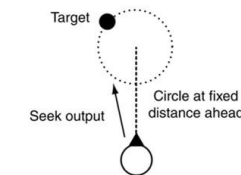
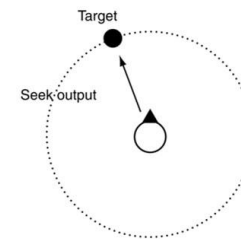
Behaviors for dynamic targets

- Variable matching is unsuited for moving targets, but is used in complex behaviors
- **Pursue / evade** tries to chase / escape from a moving target trying to predict its future position (e.g., by assuming constant velocity) and calling seek / flee (or arrive) on predicted position
- **Face** tries to look at a target calling align after determining direction



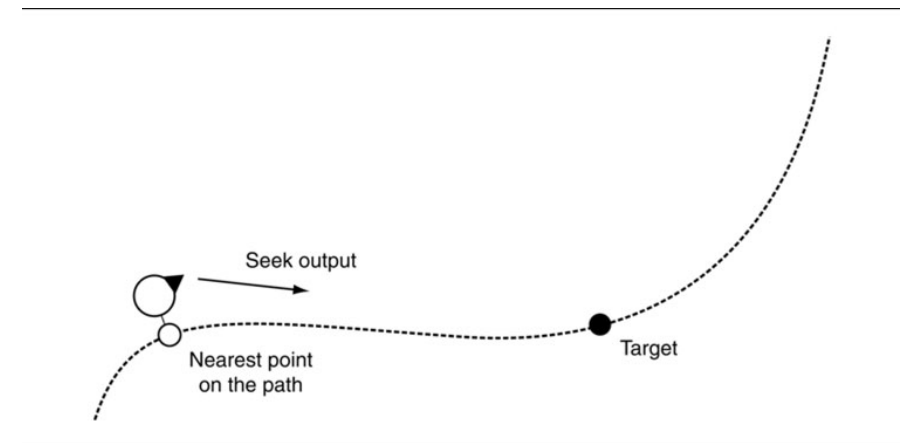
Wander

- Wander behavior controls a character moving aimlessly
- Wander can be implemented by randomly changing orientation and moving forward
- It can be improved by randomly moving a target around a circle around the character and calling seek on the target
- A further improved version moves the circle at a fixed distance ahead the character



Path following

- Path following takes a whole path as target
- Character should move along the path in one direction
- First, current character position is mapped to the nearest point along the path
- Second, a target is selected which is further along the path than the mapped point by some distance
- Third, seek is called on the target

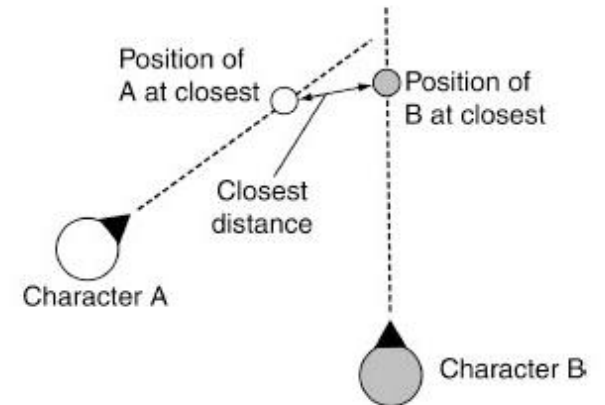
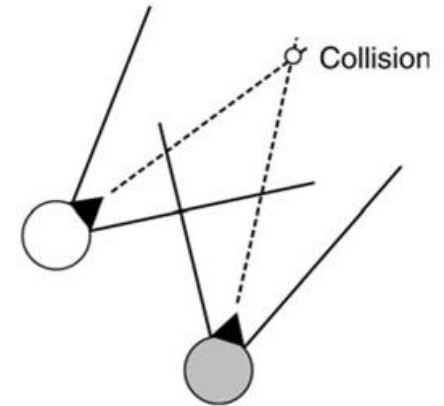
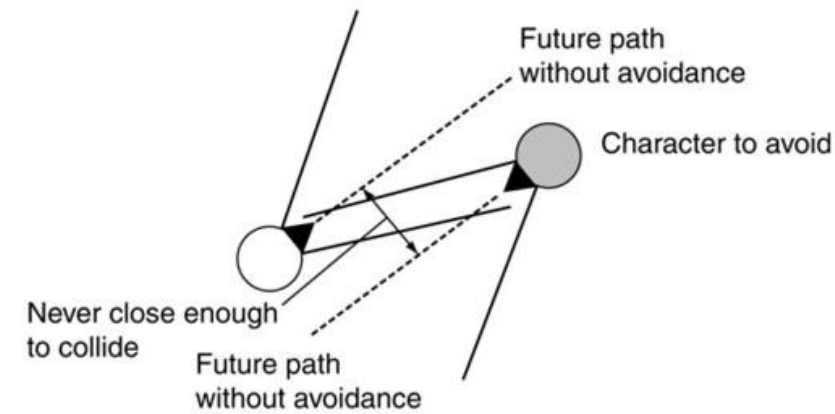


Separation

- Separation is used in crowd simulations, where several characters head in the same direction
- Most of the time, the separation behavior has a zero output
- If it detects a character closer than some threshold, it calls a variant of evade in which acceleration depends on the distance from the character
- This is an example of a steering behavior which is not much useful on its own, but in combination with others (e.g., pursue)

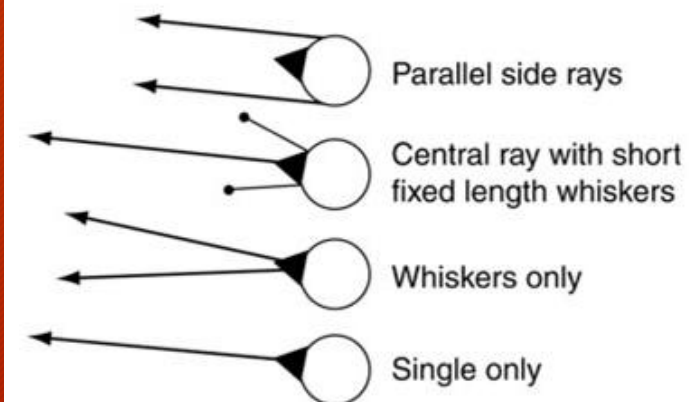
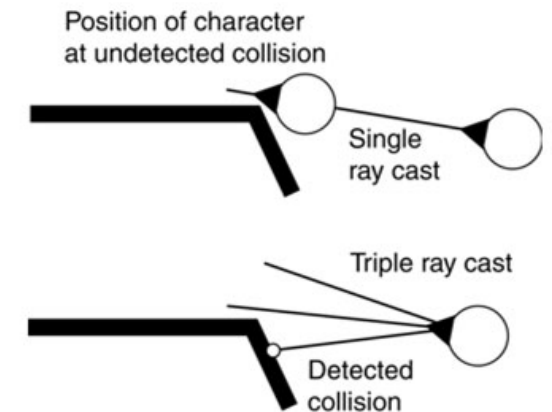
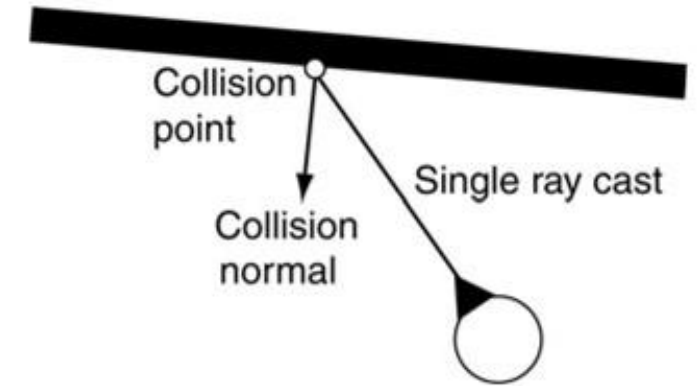
Collision avoidance

- It is useful to avoid collision with other characters crossing one character's path
- The basic approach is to use separation or evade when another character is inside a cone of view
- This approach has false positives and negatives (in fig.)
- A better approach can predict future positions (e.g., assuming same velocity)
- It assumes targets are spherical



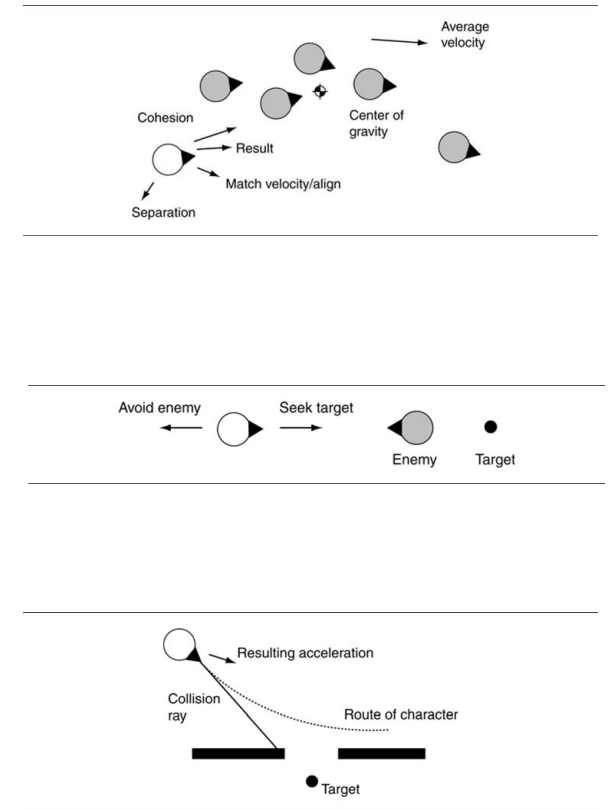
Obstacle and wall avoidance

- To avoid walls and other obstacles that are not spherical, a different behavior based on raycast can be used
- To solve some issues, more than a ray (in fig.) or even a volume can be used
- If rays collide with an obstacle, a target is created to avoid collision, and the character does a basic seek on this target



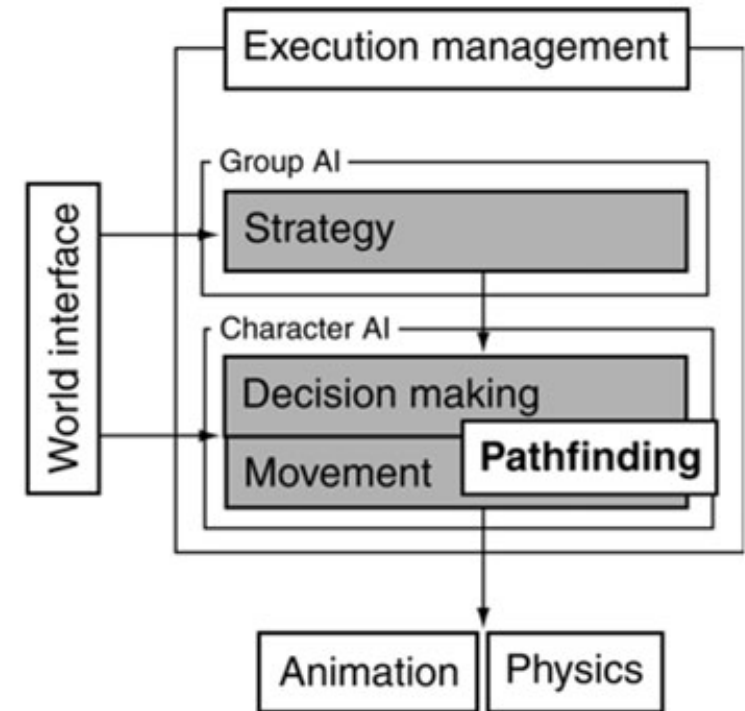
Combining steering behaviors

- In some games, one steering behavior is enough, in others multiple behaviors must be combined (e.g., seek without collisions)
- Behaviors can be combined using weighted blending, i.e., running all behaviors and combining output using weights
- It works for some behaviors, but can lead to equilibrium or to miss the target
- Most issues can be solved grouping behaviors by priorities and combining blending with arbitration



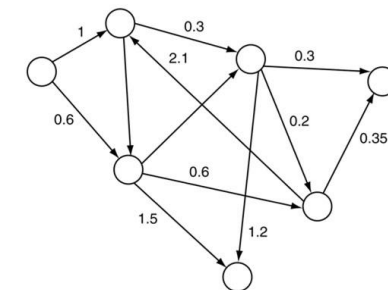
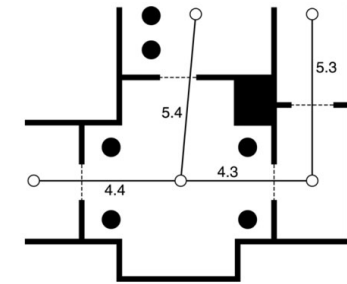
Pathfinding

- AI must be able to calculate a suitable route through the game level to get from where character is now to its goal using shortest or more rapid path
- This AI task is called pathfinding or path planning
- In the AI model, it is between movement and decision making



Pathfinding graph

- Pathfinding algorithms do not work directly on meshes, but on a graph representation
- The graph is non-negative weighted (there is a cost to go from a node to another)
- The graph is usually directed
- Pathfinding consists in finding a path from a start node to a goal node with minimal cost (or reasonably low)



Dijkstra's problem

- Dijkstra, A note on two problems in connexion with graphs, Numerische Mathematik, 1(1):269-271, 1959
- Shortest path problem, i.e., shortest routes to everywhere from an initial point (guaranteed to have the minimal cost)
- Solution to Dijkstra's problem includes solution to pathfinding problem (shortest path from start to specific goal), and the algorithm can be modified to generate only the path of interest
- In practice, it is less efficient than other approaches, which however are still based on it

Dijkstra's algorithm

- It uses a list of open nodes (i.e., to be processed) and a list of closed nodes (i.e., already processed); all the other are unvisited
- At the beginning, start is the only node in the open list
- At each iteration, it considers one node and follows outgoing arcs, storing the node at the other end in the open list
- It also stores in destination node the arc used to reach it
- Then current node is moved to the close list
- When current node is goal, the solution is following back arcs
- If the open list becomes empty, there is no solution

Dijkstra's node selection

- Current node is the one in the open list with the lowest cost-so-far
- For each node at the end of an arc of current node, cost-so-far is the cost-so-far of current node plus the cost of the arc
- Cost-so-far of start is 0
- If during an iteration we reach a node already in a list and the new cost-so-far is lower than stored one, the cost-so-far and the arc to reach it are updated, and the node goes to the open list
- Unfortunately, this strategy leads to a high **fill**, i.e., number of considered nodes

A* algorithm

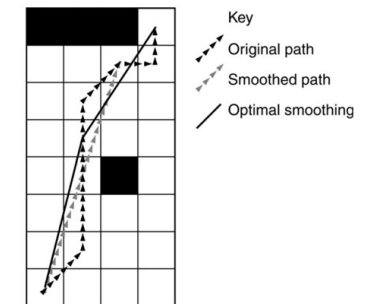
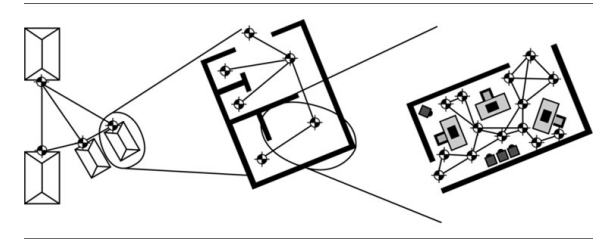
- A* is based on Dijkstra, but is specifically focused on pathfinding
- With little variations, it is the most used in games
- To reduce fill, instead of selecting nodes with lowest cost-so-far, it uses a heuristic to guess the lowest estimated-total-cost
- When visiting a node, in addition to cost-so-far and incoming arc, it calculates estimated-total-cost as the sum of the cost-so-far to reach the node and an estimate of how far it is from the goal
- In general, it does not guarantee that found path cost is minimum, but works well in most cases

A* heuristics

- Heuristics are usually parameterized with goal node
- Bad heuristics can make A* worst than Dijkstra
- Speed of heuristics can severely impact on performance
- If the heuristic is too low, so that it underestimates the actual path length, A* takes longer to run
- If the heuristic is too high, so that it overestimates the actual path length, A* may not return the best path
- A popular underestimating heuristics is **Euclidean distance**, good for outdoor environments
- A **cluster heuristics** (e.g., grouping nodes in the same room) is good for indoor environments

Improving A*

- The algorithm can be used on large scale worlds using hierarchical pathfinding (in fig.)
- The result of A* can be improved by smoothing the path (in fig.)
- When more than one goal is suited, open goal pathfinding can be used
- In dynamic pathfinding, information about the world can be updated at runtime
- ...



World representation

- A challenging task is representing game world as a graph
- Quantization is world position to graph nodes conversion
- Localization is graph path to world position conversion
- A possible representation (if world is made of tiles or based on a grid) is using tile graphs, where each tile is a node
- Most of current games (and engines) use navigation meshes (**navmesh**), a representation based on triangles or quads where each polygon is a node and each edge an arc in the graph

Bibliografy

- Millington, AI for Games, 3rd ed., CRC Press, 2019
- Rabin, Game AI Pro 360: Guide to Movement and Pathfinding, CRC Press, 2020
- Roberts, Artificial Intelligence in Games, CRC Press, 2023

